

Database Architecture

PostgreSQL schema reference — Auth & Identity, RBAC, Patients, Queue & Scheduling

Generated	August 16, 2026
Source	db/migrations/V1 – V5
Tables	12
Engine	PostgreSQL (citext, pg_trgm extensions)
Multi-tenancy	Shared schema, Row-Level Security

This document reflects the schema exactly as committed across five migrations — nothing here is aspirational. It covers what's actually built: staff authentication and sessions, RBAC, patient records, and queue/appointment scheduling. Later backlog epics (Caregiver/Family, Messaging Platform, Patient WhatsApp Surface, Pharmacy, and others) are not yet built and are not shown.

Architecture Overview

Multi-tenancy: shared schema + Row-Level Security

Every tenant-scoped table carries a `tenant_id` column and a Postgres Row-Level Security (RLS) policy: `USING (tenant_id = current_setting('app.tenant_id', true)::uuid)`. The application sets that session variable once per request via `TenantContext`, right after the tenant is resolved. RLS is the second layer, not the only one — every repository query also filters by `tenant_id` explicitly. Two tables opt out deliberately:

- **tenants** itself has no RLS — resolving *which* tenant a request belongs to has to happen before any tenant context exists.
- **login_attempts** has no RLS — it's append-only telemetry, always queried pre-scoped by `staff_id / email / ip_address`, never listed raw by tenant.

`tenants.region` is locked by a `BEFORE UPDATE` trigger the moment it's set at provisioning — an attempted change raises an exception rather than silently succeeding.

The RLS bootstrap problem, and how it's solved

Three flows need to read a tenant-scoped table *before* the tenant is known: refresh-token rotation, staff-invite acceptance, and (implicitly) anything keyed by a bearer secret rather than an authenticated session. Each is solved the same way — a narrow `SECURITY DEFINER` SQL function that steps outside RLS for exactly that one lookup, on the reasoning that possessing an unguessable 256-bit token *is* the authorization, the same trust model as any bearer credential:

- `find_session_by_token_hash(text)` — refresh-token rotation (V1)
- `find_staff_by_invite_token_hash(text)` — invite acceptance (V2)

Every other query in the codebase goes through the ordinary RLS-scoped path via the app's non-owner database role.

Concurrency guarantees enforced by the database, not application code

Two places need to survive real concurrent requests, and both are backed by Postgres primitives rather than a check-then-write race:

- **No double-booking** — a partial unique index, `UNIQUE (doctor_id, start_time) WHERE status='scheduled'`, on `appointments`. Verified under 10 truly concurrent booking requests for the same slot: exactly one 201, nine 409 s.
- **Unique daily queue tokens** — a Postgres advisory lock (`pg_advisory_xact_lock`) keyed by `(doctor_id, date)` serializes token assignment; the same lock also serializes the session-capacity check in `doctor_working_hours.max_patients`, since scheduled and walk-in patients draw from one shared pool. Verified under 8 concurrent check-ins: tokens 1–8, no duplicates.

Encryption & secret handling

WHAT	HOW	WHERE
Staff PIN	Argon2id (OWASP params: m=19MiB, t=2, p=1)	<code>staff.pin_hash</code>
Refresh / invite / WhatsApp OTP tokens	SHA-256 of a random secret; only the hash is stored, raw value shown once	<code>sessions.token_hash</code> , <code>staff.invite_token_hash</code> , <code>staff.whatsapp_otp_hash</code>
TOTP MFA secret	AES-256-GCM, app-held key	<code>staff.mfa_secret_enc</code>
National ID (Aadhaar/Iqama)	AES-256-GCM, app-held key; never returned by any API response	<code>patients.national_id_enc</code>

Consent and row-level scoping — enforced in the query, not the UI

`consents.withdrawn_at` being set removes a patient from every read path immediately — the filter lives in `PatientRepository`'s SQL, not in a response filter. Similarly, a doctor whose `staff.scope = 'own_patients_only'` only sees patients they have an `appointments` row with; a direct-ID request for a patient outside that scope returns `404` , never `403` — indistinguishable from the patient not existing at all.

Migration History

FILE	ADDS	BACKLOG REFS
V1__staff_auth_core	<code>tenants</code> , <code>roles</code> , <code>staff</code> , <code>sessions</code> , <code>login_attempts</code> — login, MFA, refresh-token rotation with reuse detection, lockout backoff	NB-002, NB-003, NB-029, NB-032, NB-039
V2__staff_roles_endpoints	<code>staff.field_grants</code> ; <code>find_staff_by_invite_token_hash()</code> bootstrap function	NB-039 – NB-048
V3__patients	<code>patients</code> — registration, MRN sequence, trigram search, soft-merge scaffold	NB-059 – NB-062
V4__queue_appointments	<code>doctor_working_hours</code> , <code>doctor_leave</code> , <code>appointments</code> , <code>queue_entries</code> — slot generation, double-booking guard, queue state machine	NB-067 – NB-069, NB-072, NB-074
V5__v2_plan_sync	Region-lock trigger; <code>staff</code> PIN/OTP/verification columns (drops <code>password_hash</code>); <code>consents</code> ; <code>patient_merges</code> ; <code>doctor_working_hours.max_patients</code> ; queue status split into <code>vitals_pending</code> / <code>vitals_done</code>	v2 plan sync — NB-002, NB- 040/041/046, NB- 051, NB-053, NB- 072, NB-094, NB- 098

Deliberately deferred (marked `ponytail:` in the migrations)

- No purge job for expired/revoked `sessions` or old `login_attempts` rows — fine at current volume, add a nightly delete once it isn't.
- Region-change is *rejected* by the trigger but not yet *audited* — there's no audit-trail table to write to (that's a separate, not-yet-built epic).
- `consents` models withdrawal only, not capture-at-registration — a patient with no consent rows is treated as not-withdrawn, not as unconsented.
- `doctor_leave` blocks the whole day and doesn't cascade to flag existing bookings for reschedule — no notification channel exists yet to flag through.
- Row-level "own patients only" scoping uses `appointments` as the ownership signal, a proxy for the real Encounters table that Clinical Workspace (not yet built) will introduce.

Entity-Relationship Diagram

Identity & Tenancy Patients Queue & Scheduling required FK nullable / optional FK

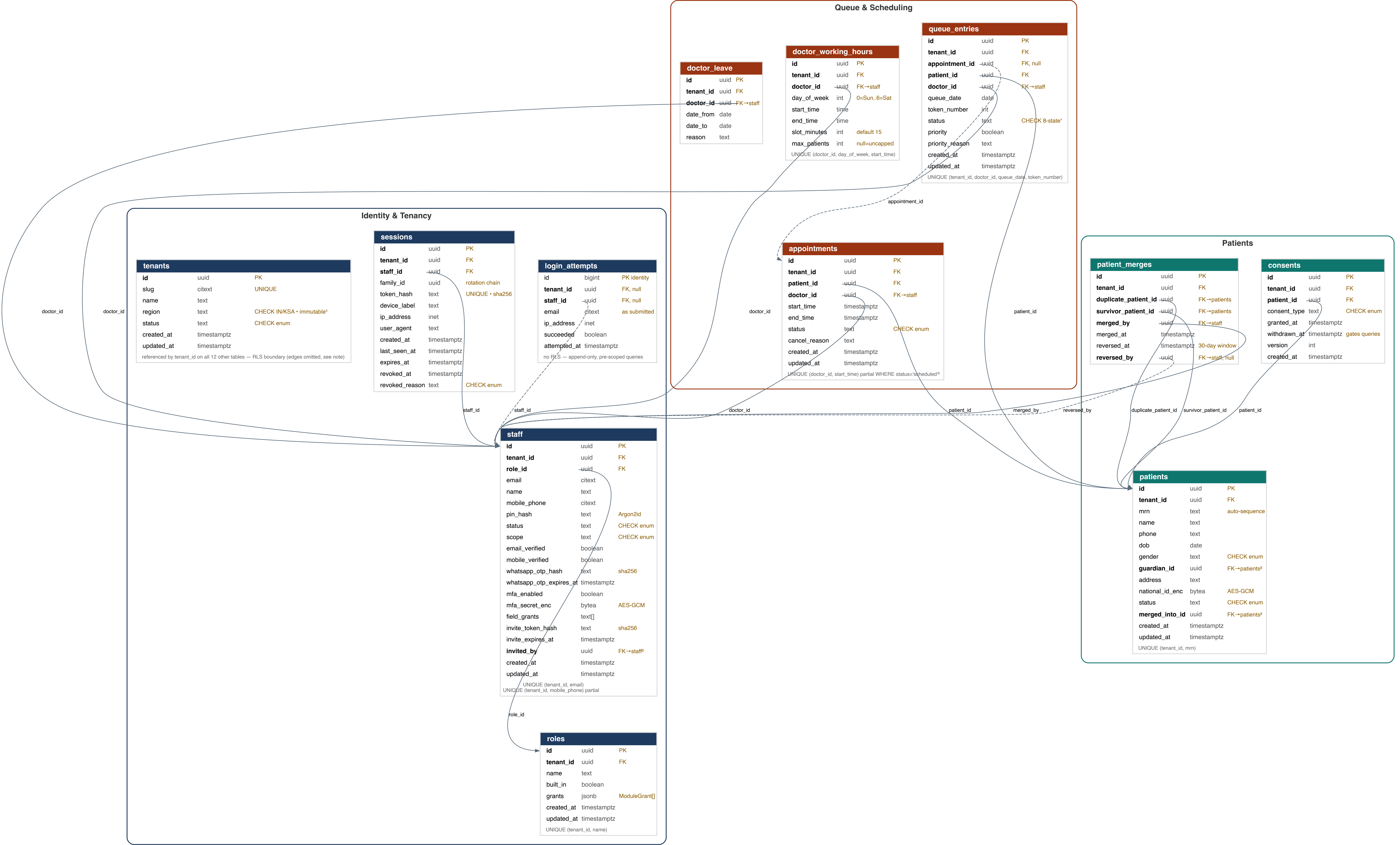


Table Reference

TABLE	MODULE	PURPOSE
tenants	Identity	One row per clinic. <code>region</code> is immutable after provisioning (NB-002).
roles	Identity	Built-in and custom RBAC roles; <code>grants</code> is a <code>ModuleGrant[]</code> JSON array (module × action).
staff	Identity	Clinic staff. PIN + WhatsApp-OTP + authenticator-app MFA login (no passwords, no SSO — NFR-06). Dual-channel <code>email_verified</code> / <code>mobile_verified</code> gate patient-data access.
sessions	Identity	Refresh-token rotation chain (<code>family_id</code>); reused/stale token revokes the whole chain.
login_attempts	Identity	Append-only lockout/rate-limit telemetry — no separate failure counter column, computed from this log.
patients	Patients	Patient records. Auto MRN, trigram-indexed for typo-tolerant search, encrypted national ID, soft-merge via <code>status</code> / <code>merged_into_id</code> .
consents	Patients	Consent withdrawal events; a withdrawn <code>data_processing</code> consent removes the patient from every read path (NB-053).
patient_merges	Patients	Merge audit log; reversible for 30 days via <code>reversed_at</code> / <code>reversed_by</code> (NB-072).
doctor_working_hours	Queue	Weekly recurring availability template; drives slot generation and the shared <code>max_patients</code> session cap (NB-098).
doctor_leave	Queue	Date-range blocks; any overlap blocks the entire day.
appointments	Queue	Booked slots. Double-booking prevented by a DB-level partial unique index, not application logic.
queue_entries	Queue	Today's walk-through queue. <code>appointment_id</code> null = walk-in. 8-state machine, illegal transitions rejected.

Diagram footnotes

¹ `tenants` is referenced by `tenant_id` on all 12 other tables (the RLS boundary) — those edges are omitted from the diagram; drawing all twelve added no information, only clutter.

² Self-referential foreign key (`staff.invited_by` → `staff.id` , `patients.guardian_id` → `patients.id` , `patients.merged_into_id` → `patients.id`) — drawn as a note, not a loop edge, for the same reason.

³ `appointments`: `UNIQUE (doctor_id, start_time) WHERE status='scheduled'` — a *partial* index, so a cancelled slot immediately frees the time for a new booking. This is the mechanism that makes double-booking prevention safe under real concurrency.

⁴ `queue_entries.status`: `checked_in` → `waiting` → `vitals_pending` → `vitals_done` → `in_consult` → `checkout_pending` → `completed` , with `no_show` reachable from any state before checkout. Enforced in `QueueService` , not just documented — illegal transitions return `400` .